

CUSTOM CONTENT GENERATOR · IMPLEMENTATION REVIEW

Technical documentation

A source-grounded overview of the generator’s architecture, AI/API choices, prompt design, performance measures, and operational limits.

Prepared against the current application code and the attached Practical Project brief. Recommendations are identified separately from implemented behavior.

01 · Implementation architecture

EcoFlow Studio creates sustainable-product marketing copy. Users select a content format, tone, length, and target audience, then provide a product name and optional features and keywords. The five formats are product descriptions, Instagram captions, email subject lines, taglines, and SEO descriptions. The interface also supports batch generation, saved history, usage statistics, and copy/export actions.



The project is a pnpm TypeScript workspace. The browser client uses React 19, Vite, Wouter routing, TanStack Query, React Hook Form, and Zod. An Express 5 service exposes JSON endpoints. PostgreSQL stores generations; Drizzle ORM defines and queries the schema. The OpenAPI 3.1 document is the API contract, and Orval generates client hooks and server-side Zod schemas from it.

Request lifecycle: the browser submits a form to `/api/generate`; the API validates the payload, builds a prompt, calls `gpt-5-mini`, stores a successful response, and returns content plus timing/token metadata. The UI refreshes affected history and statistics after a successful mutation. Batch generation runs the five format requests concurrently.

LAYER	ROLE
Web client	Form, result panel, batch, history, and stats experiences.
API + contract	Express routes; OpenAPI contract; generated hooks and request schemas.
Data + AI	Drizzle/PostgreSQL persistence; OpenAI-compatible integration client.

02 · API selection rationale

OpenAI through Replit AI Integrations. The implementation calls `gpt-5-mini` using chat-completion messages and caps completions at 1,024 tokens. This fits short marketing-copy tasks and supports separate role and task instructions. The existing integration keeps provider credentials out of application source. This is a fit-to-implementation rationale, not a comparative benchmark claim.

PostgreSQL + Drizzle suit structured generation records and SQL aggregation for history and usage views. **OpenAPI + Orval + TanStack Query** reduce client/server contract drift and provide mutation state and targeted cache refresh without polling.

API surface: health check; single and batch generation; history list and deletion; usage statistics. History accepts limit, offset, and content-type filters.

02 · PROMPT ENGINEERING METHODOLOGY

Task-specific prompts, shared inputs

Each generation uses a system message and a content-specific user prompt. For single generation, the system message establishes an expert sustainable-marketing-copywriter persona and asks for authentic, benefits-focused writing. Batch generation uses a shorter version of that role instruction; aligning both versions is a straightforward consistency improvement.

The server's buildPrompt function combines one of five templates with product name, optional features and keywords, tone, an audience description, and an explicit length guide. Length values map to short (20–50 words), medium (50–120 words), and long (150–300 words). Audience codes are mapped to plain-language descriptions. Optional feature/keyword instructions are added only when those fields are present.

TEMPLATE	SPECIFIC OUTPUT GUIDANCE
Product description	Benefits, sustainability angle, and product distinction; output copy only.
Instagram caption	Strong opening hook, shareable tone, and 3–5 relevant hashtags.
Email subject lines	Five numbered options under 60 characters; include curiosity, urgency, and benefit-led approaches.
Taglines	Five numbered memorable options, each under 10 words.
SEO description	150–160 characters, natural primary keyword, and a call to action.

Output-only instructions reduce prefatory text and unwanted labels. They remain model instructions, not guarantees; generated length and format need validation if downstream systems depend on them.

Input validation and output filtering

The client applies React Hook Form and Zod checks for required values. At the API boundary, generated Zod schemas validate the request against the OpenAPI contract; content type, tone, length, and audience are enums. Malformed requests return HTTP 400 before inference. Free-text fields do not currently have explicit maximum lengths.

Current gap: there is no separate moderation step, prompt-injection defense, or post-generation output filter. The model response is persisted and displayed as returned. Therefore, input shape validation is implemented, but independent output filtering is not.

Recommended safeguards for production

- Set maximum lengths for product name, features, and keywords; treat user text as data, not instructions.
- Reject or flag empty responses and run type-specific checks, such as subject-line count and SEO character length.
- Apply appropriate moderation, and route environmental, health, or compliance claims to human review.
- Report a validation warning instead of silently changing generated claims or meaning.

Method summary: stable role → format-specific objective and constraints → mapped user settings → explicit output format. Keep templates independently testable so prompt revisions can be compared for relevance, factuality, format compliance, and token use.

03 · RELIABILITY, PERFORMANCE, AND LIMITATIONS

Performance optimization techniques

- 1. **Concurrent batch generation.** Promise.all runs the five format requests concurrently. Ideal wall time approaches the slowest call rather than the sum of five calls, but peak provider demand rises.
- 2. **Validate before inference.** Invalid requests are rejected before the model call, avoiding avoidable latency and token use.
- 3. **Completion ceiling.** A 1,024-token maximum bounds each completion; it does not limit prompt size or directly estimate spend.
- 4. **Insert and return.** A completed result is inserted and returned in one database round trip, avoiding a follow-up read.
- 5. **SQL-side statistics.** Totals, averages, type/tone counts, and a 14-day activity series are aggregated in PostgreSQL rather than in application memory.
- 6. **Targeted refresh.** Successful generation invalidates history and stats queries; the app does not continuously poll those views.

Limitation management strategies

LIMIT	CURRENT STATE AND MANAGEMENT DIRECTION
Rate limits / outages	Errors are logged and returned as generic HTTP 500 responses. There is no retry, queue, or application-level rate limiter. Add bounded concurrency, 429-aware exponential backoff with jitter, and clear retry guidance.
Batch partial failure	Concurrent inserts may persist even if another request rejects and the endpoint returns an error. Return per-format outcomes or track batch status; make retries idempotent to avoid duplicates.
Latency	Calls are synchronous; users wait for completion while a loading state is shown. Measure p50/p95 latency before adding streaming, which increases transport and error-handling complexity.
Safety / quality	No independent input moderation, output filter, or claim verification exists. Add input limits, format checks, moderation where appropriate, and human review for consequential claims.
Data isolation	Generation rows have no user ownership field; history and stats are not account-scoped. Treat as single-tenant until authentication, ownership, authorization, and migration are implemented.

Rate limits and usage costs

The API stores total tokens and generation duration, but not separate input/output token counts, a model-version field, provider quota values, or dollar cost. No fixed rate-limit or price figure is embedded in the app because model pricing and account quotas can change and may depend on the active integration.

Operational practice: before launch, verify the active model's current provider pricing and account limits. For cost reporting, store input and output token counts plus model identifier, version the applicable price table, and calculate an estimate—not an invoice. Monitor request volume, 429/5xx rates, latency percentiles, token totals, and estimated spend; set alerts and review limits when the model or plan changes.

The 1,024-token ceiling and recorded usage are guardrails and telemetry. They do not guarantee availability, prevent aggregate overuse, or replace provider-side quota monitoring.

Source basis: current EcoFlow Studio API routes, OpenAPI specification, Drizzle schema, and generator UI, checked against the attached Practical Project brief. Proposed controls above are not described as shipped features.